

Assignment 8

Q1. Student Result Processing (reduce + Classes)

You are given an array of marks of a student.

Create a Student class with properties: name and marks[].

Add method:

- calculateAverage() using reduce()
- Return grade based on average: A/B/C/F

Test the class for 3 students.

Q2. Online Food Ordering (map + Error Handling)

You have an array of menu items with prices.

Write a function calculateBill(orderItems) that:

- Uses map() to get price list
- Uses reduce() to calculate the total
- Throws an error if an invalid item is ordered
- Use try-catch to show meaningful messages

Q3. Product Discount System

Create a Product constructor function with:

- name
- price

Add a prototype method applyDiscount(percent) that returns new price.

Now create 3 products and apply different discounts.

Show how abstraction helps simplify operations.

Q4. Employee Inheritance

Create classes Employee and Manager.

- Employee → properties: name, department
- Manager → inherits Employee and overrides a work () method

Create objects and show runtime polymorphism.

Q5. Ride-Sharing Application

Create classes:

- User → name, rating
- Driver → inherits from User, vehicle details
- Trip → fromLocation, toLocation, distance

Scenario:

- Write a calculateFare () method inside Trip using distance.
- Throw an error if distance is negative or not provided.
- Use try/catch to handle the error and show a proper message.

Q6. E-Commerce Inventory System

You have an array of product objects with:

```
{ id, name, category, price, stock }
```

Write functions using array methods:

1. getLowStockProducts () → using filter ()
2. sortProductsByPrice () → using sort ()
3. calculateTotalInventoryValue () → using reduce ()
4. groupByCategory () → using reduce () + object grouping

Test using multiple category products.

Q7. Banking Application

Create a BankAccount class with private fields:

- #balance
- Add methods:

- `deposit(amount)`
- `withdraw(amount)` (throw error if insufficient balance)
- `getBalance()`

Demonstrate:

- valid deposits
- invalid withdrawals (handled using try/catch)

Q8. Movie Ticket Booking System

Create a class `MovieTicket` with properties:

- `movieName`
- `seatNo`
- `price`

Then create a class `OnlineTicket` that inherits and adds:

- `convenienceFee`
- method `getTotalAmount()`

Additionally:

- Add a method to `MovieTicket.prototype` called `printTicket()`
- Call it using `OnlineTicket` objects (demonstrate prototype chain working)

Q9. Fitness App Analytics

Given user workout data:

```
[
  { user: "A", steps: 8000, calories: 300 },
  { user: "B", steps: 12000, calories: 500 },
  { user: "C", steps: 4000, calories: 200 }
]
```

Create a `FitnessAnalytics` class with methods:

- `getActiveUsers()` → users with steps > 7000 using `filter()`
 - `getAverageCalories()` → using `reduce()`
 - `getUserSummary()` → using `map()` to format as messages
 - Throw an error if dataset is empty
- Execute all methods for the above data.